

SQL — Complete Reference

EY Probation Training — Week 2. Scope: fundamentals, joins, aggregations, subqueries, window functions, CTEs. Dataset: AdventureWorks.

This is the highest bar in the whole roadmap. Every other outcome says “understand” or “gain familiarity”; this one says **“able to independently write and optimize SQL queries.”** Prioritise accordingly.

1. Logical Execution Order

Not the order you write it. This explains most SQL confusion.

```
1. FROM / JOIN      -> build the working set
2. WHERE            -> filter individual ROWS
3. GROUP BY         -> collapse into groups
4. HAVING           -> filter GROUPS
5. SELECT           -> compute output columns
6. DISTINCT
7. ORDER BY         -> sort
8. LIMIT / TOP      -> truncate
```

Consequences you must be able to state: - **You cannot use a `SELECT` alias in `WHERE`** — `WHERE` runs before `SELECT`. (Most engines DO allow it in `ORDER BY`, which runs after.) - **`WHERE` filters rows, `HAVING` filters groups.** `WHERE` cannot contain an aggregate; `HAVING` normally does. - **Window functions cannot appear in `WHERE` or `HAVING`** — they’re computed in step 5, after both. Wrap in a CTE or subquery first. This is a favourite exam question.

2. Basics

```
SELECT DISTINCT c.FirstName, c.LastName
FROM Person.Person AS c
WHERE c.LastName LIKE 'S%' AND c.EmailPromotion > 0
ORDER BY c.LastName ASC, c.FirstName DESC
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY;    -- ANSI; SQL Server also has TOP 10
```

Operators: `=` `<` `< >` `<=` `>=`, `BETWEEN a AND b` (inclusive), `IN (...)`, `LIKE`, `IS NULL`, `IS NOT NULL`, `AND` / `OR` / `NOT`.

Wildcards: `%` = any number of characters, `_` = exactly one.

NULL semantics — heavily tested

- `NULL = NULL` is **not true**; it’s unknown. Always use `IS NULL`.
- Any arithmetic with `NULL` yields `NULL`: `100 + NULL = NULL`.
- `NOT IN (SELECT ...)` returns **no rows** if the subquery contains a single `NULL`. Use `NOT EXISTS` instead — this is a real production bug, not just trivia.
- `COUNT(*)` counts rows; `COUNT(col)` **skips NULLs**. `COUNT(DISTINCT col)` also skips them.
- `AVG(col)` divides by the count of non-`NULL` values, not by the row count.

```
COALESCE(a, b, c)      -- first non-null, ANSI standard
ISNULL(a, 0)           -- SQL Server, two args only
NULLIF(a, b)           -- NULL if a = b; classic divide-by-zero guard:
SELECT total / NULLIF(count, 0) FROM t;
```

CASE

```
SELECT OrderID,
CASE WHEN Total >= 10000 THEN 'Large'
     WHEN Total >= 1000  THEN 'Medium'
     ELSE 'Small' END AS Band,
CASE Status WHEN 1 THEN 'New' WHEN 5 THEN 'Shipped' ELSE 'Other' END AS StatusName
FROM Sales.SalesOrderHeader;
```

Evaluated top-down, first match wins. `ELSE` omitted means unmatched rows get `NULL`.

Conditional aggregation — the pivot idiom, worth memorising:

```
SELECT TerritoryID,
SUM(CASE WHEN YEAR(OrderDate) = 2013 THEN TotalDue ELSE 0 END) AS Rev2013,
SUM(CASE WHEN YEAR(OrderDate) = 2014 THEN TotalDue ELSE 0 END) AS Rev2014,
COUNT(CASE WHEN OnlineOrderFlag = 1 THEN 1 END) AS OnlineOrders
FROM Sales.SalesOrderHeader GROUP BY TerritoryID;
```

Note `COUNT(CASE ... THEN 1 END)` with no `ELSE`: unmatched rows become NULL and `COUNT` skips them. Using `ELSE 0` there would count everything.

3. Joins

```
SELECT h.SalesOrderID, d.ProductID, p.Name
FROM Sales.SalesOrderHeader h
INNER JOIN Sales.SalesOrderDetail d ON h.SalesOrderID = d.SalesOrderID
LEFT JOIN Production.Product p ON d.ProductID = p.ProductID
WHERE h.OrderDate >= '2013-01-01';
```

Join	Returns
INNER	Only matching rows from both
LEFT [OUTER]	All left rows; right columns NULL when unmatched
RIGHT [OUTER]	All right rows
FULL [OUTER]	All rows from both
CROSS	Cartesian product
SELF	A table joined to itself with aliases

The LEFT JOIN + WHERE trap

```
-- BROKEN: this silently becomes an INNER join
SELECT c.CustomerID, o.SalesOrderID
FROM Customer c LEFT JOIN SalesOrderHeader o ON c.CustomerID = o.CustomerID
WHERE o.OrderDate >= '2014-01-01';

-- CORRECT: put the right-table condition in the ON clause
FROM Customer c LEFT JOIN SalesOrderHeader o
  ON c.CustomerID = o.CustomerID AND o.OrderDate >= '2014-01-01';
```

Unmatched rows have `o.OrderDate = NULL`, and `NULL >= '2014-01-01'` is unknown, so `WHERE` discards them. **Conditions on the right table of a LEFT JOIN belong in ON; conditions on the left table belong in WHERE.**

Finding non-matches

```
SELECT c.CustomerID FROM Customer c
LEFT JOIN SalesOrderHeader o ON c.CustomerID = o.CustomerID
WHERE o.CustomerID IS NULL; -- anti-join idiom

SELECT CustomerID FROM Customer c
WHERE NOT EXISTS (SELECT 1 FROM SalesOrderHeader o
  WHERE o.CustomerID = c.CustomerID); -- usually preferred
```

Self-join

```
SELECT e.BusinessEntityID, e.JobTitle, m.JobTitle AS ManagerTitle
FROM HumanResources.Employee e
LEFT JOIN HumanResources.Employee m ON e.ManagerID = m.BusinessEntityID;
```

For pair-wise self-joins use `a.id < b.id` rather than `<>` — otherwise every pair appears twice.

Join fan-out: if the right table's key isn't unique, rows multiply and every `SUM` is inflated. Check `SELECT key, COUNT(*) FROM dim GROUP BY key HAVING COUNT(*) > 1` before joining, and verify the row count is unchanged after.

4. Aggregation

```

SELECT TerritoryID,
       COUNT(*)           AS Orders,
       COUNT(DISTINCT CustomerID) AS Customers,
       SUM(TotalDue)       AS Revenue,
       AVG(TotalDue)       AS AvgOrder,
       MIN(OrderDate)     AS FirstOrder,
       MAX(TotalDue)       AS LargestOrder
FROM Sales.SalesOrderHeader
WHERE OrderDate >= '2013-01-01' -- filters ROWS, before grouping
GROUP BY TerritoryID
HAVING SUM(TotalDue) > 1000000 -- filters GROUPS, after
ORDER BY Revenue DESC;

```

Rule: every non-aggregated column in `SELECT` must appear in `GROUP BY`. (MySQL is lenient by default; standard SQL and SQL Server are not.)

```

GROUP BY ROLLUP(Region, City) -- hierarchical subtotals + grand total
GROUP BY CUBE(Region, Channel) -- every combination
GROUP BY GROUPING SETS ((Region), (Channel), ())

```

5. Subqueries

Scalar — returns one value:

```

SELECT Name, ListPrice,
       (SELECT AVG(ListPrice) FROM Production.Product) AS AvgPrice
FROM Production.Product
WHERE ListPrice > (SELECT AVG(ListPrice) FROM Production.Product);

```

Multi-row with `IN` / `ANY` / `ALL` :

```

WHERE TerritoryID IN (SELECT TerritoryID FROM Sales.SalesTerritory WHERE [Group] = 'Europe')
WHERE ListPrice > ALL (SELECT ListPrice FROM Product WHERE Colour = 'Red')

```

Correlated — references the outer query, conceptually re-evaluated per row:

```

SELECT p.Name, p.ListPrice
FROM Production.Product p
WHERE p.ListPrice > (SELECT AVG(p2.ListPrice) FROM Production.Product p2
                   WHERE p2.ProductSubcategoryID = p.ProductSubcategoryID);

```

EXISTS — stops at the first match, so it's usually faster than `IN` and is NULL-safe:

```

WHERE EXISTS (SELECT 1 FROM SalesOrderDetail d WHERE d.ProductID = p.ProductID)
WHERE NOT EXISTS (...) -- prefer over NOT IN

```

Derived table (subquery in `FROM`) — must be aliased:

```

SELECT t.TerritoryID, t.Revenue
FROM (SELECT TerritoryID, SUM(TotalDue) AS Revenue
      FROM Sales.SalesOrderHeader GROUP BY TerritoryID) AS t
WHERE t.Revenue > 1000000;

```

6. CTEs

```

WITH TerritoryRevenue AS (
  SELECT TerritoryID, SUM(TotalDue) AS Revenue
  FROM Sales.SalesOrderHeader GROUP BY TerritoryID
),
Ranked AS (
  SELECT *, RANK() OVER (ORDER BY Revenue DESC) AS rnk
  FROM TerritoryRevenue
)
SELECT * FROM Ranked WHERE rnk <= 5;

```

- Readable top-to-bottom instead of inside-out.
- A later CTE can reference an earlier one — this is how you filter on a window function.
- Referenced multiple times in one query without repeating the text.

- A CTE is **not** automatically materialised; it's usually inlined by the optimiser. It is not a performance feature per se.

Recursive CTE — for hierarchies (org charts, BOM explosions):

```
WITH OrgChart AS (
    SELECT BusinessEntityID, ManagerID, 0 AS Depth      -- anchor
    FROM HumanResources.Employee WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.BusinessEntityID, e.ManagerID, oc.Depth + 1 -- recursive part
    FROM HumanResources.Employee e
    JOIN OrgChart oc ON e.ManagerID = oc.BusinessEntityID
)
SELECT * FROM OrgChart OPTION (MAXRECURSION 100);
```

Structure: anchor member, `UNION ALL`, recursive member referencing the CTE. Always bound the recursion.

7. Window Functions

```
function() OVER (
    PARTITION BY col      -- optional: reset per group
    ORDER BY col          -- required for ranking and offsets
    ROWS BETWEEN ... AND ... -- optional frame
)
```

Unlike `GROUP BY`, windows **do not collapse rows** — every input row is returned with an extra computed column.

Ranking

Function	100, 90, 90, 80 →
<code>ROW_NUMBER()</code>	1, 2, 3, 4 (ties broken arbitrarily)
<code>RANK()</code>	1, 2, 2, 4 (ties share, then skip)
<code>DENSE_RANK()</code>	1, 2, 2, 3 (ties share, no skip)
<code>NTILE(4)</code>	Splits into 4 near-equal buckets
<code>PERCENT_RANK()</code>	$(\text{rank}-1)/(\text{n}-1)$

Offset

```
LAG(Revenue, 1) OVER (PARTITION BY Territory ORDER BY Yr)
LEAD(Revenue, 1, 0) OVER (...)      -- third arg = default when absent
FIRST_VALUE(x) OVER (...)
LAST_VALUE(x) OVER (... ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

LAST_VALUE trap: the default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, so `LAST_VALUE` returns the *current* row unless you specify `UNBOUNDED FOLLOWING`. Identical to the Spark trap — same spec, same bug.

Aggregate windows

```
SUM(TotalDue) OVER (PARTITION BY TerritoryID)      -- group total on every row
SUM(TotalDue) OVER (PARTITION BY T ORDER BY D
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) -- running total
AVG(TotalDue) OVER (ORDER BY D ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) -- 3-row moving avg
TotalDue * 100.0 / SUM(TotalDue) OVER (PARTITION BY TerritoryID)      -- share of group
```

ROWS vs RANGE : `ROWS` counts physical rows; `RANGE` groups by the ordering *value*, so tied values are all included in the same frame. Default is `RANGE`, which surprises people when there are ties.

Canonical patterns

```
-- Top N per group
WITH r AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY Cat ORDER BY Rev DESC) rn FROM t)
SELECT * FROM r WHERE rn <= 3;

-- Period-over-period growth
SELECT Yr, Rev,
       Rev - LAG(Rev) OVER (ORDER BY Yr) AS Change,
       100.0*(Rev - LAG(Rev) OVER (ORDER BY Yr)) / LAG(Rev) OVER (ORDER BY Yr) AS PctChange
FROM Yearly;

-- Deduplicate keeping the latest
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY ID ORDER BY UpdatedAt DESC) rn FROM t)
SELECT * FROM d WHERE rn = 1;

-- Rows above their group average
WITH a AS (SELECT *, AVG(Price) OVER (PARTITION BY Cat) AS CatAvg FROM Product)
SELECT * FROM a WHERE Price > CatAvg;
```

That last one is the answer to “you can’t put a window function in `WHERE`” — the CTE layer is mandatory.

8. Set Operations

```
SELECT a FROM t1 UNION      SELECT a FROM t2; -- deduplicates (slower)
SELECT a FROM t1 UNION ALL  SELECT a FROM t2; -- keeps duplicates (faster) – default choice
SELECT a FROM t1 INTERSECT SELECT a FROM t2;
SELECT a FROM t1 EXCEPT   SELECT a FROM t2; -- MINUS in Oracle
```

Column count and compatible types must match. `ORDER BY` goes once, at the very end.

9. Common Functions

```
-- Strings
LEN(s) / LENGTH(s); LEFT(s,n); RIGHT(s,n); SUBSTRING(s, start, len)
UPPER(s); LOWER(s); TRIM(s); LTRIM(s); RTRIM(s)
REPLACE(s, old, new); CONCAT(a, b); a + b (SQL Server) / a || b (ANSI)
CHARINDEX(sub, s) / POSITION(sub IN s)

-- Dates (SQL Server flavour)
GETDATE(); DATEADD(day, 7, d); DATEDIFF(day, a, b)
YEAR(d); MONTH(d); DAY(d); DATEPART(quarter, d); DATENAME(month, d)
EOMONTH(d); CAST(d AS DATE); FORMAT(d, 'yyyy-MM')

-- Numeric
ROUND(x, 2); CEILING(x); FLOOR(x); ABS(x); POWER(x, 2); SQRT(x)

-- Conversion
CAST(x AS DECIMAL(10,2)); CONVERT(VARCHAR, d, 23); TRY_CAST(x AS INT)
```

10. Optimization

The word “optimize” is in your outcome statement. Know these.

Indexes. A clustered index defines the physical row order (one per table); non-clustered indexes are separate structures with pointers. A **covering index** includes all columns a query needs, avoiding a lookup back to the table.

Sargability — a predicate is “sargable” if an index can be used. Wrapping the column in a function kills index usage:

```
WHERE YEAR(OrderDate) = 2014 -- NOT sargable
WHERE OrderDate >= '2014-01-01' AND OrderDate < '2015-01-01' -- sargable

WHERE LEFT(Name, 3) = 'Bik' -- NOT sargable
WHERE Name LIKE 'Bik%' -- sargable ('%Bik%' is NOT – leading wildcard)
```

The rule: never wrap the indexed column in a function. Transform the constant instead.

Other principles: - `SELECT` only the columns you need — never `SELECT *` in production. - Filter as early and as selectively as possible. - `EXISTS` over `IN` for large subqueries; `NOT EXISTS` over `NOT IN` always (NULL safety). - `UNION ALL` over `UNION` unless you actually need dedup. - Beware implicit type conversion — comparing a `VARCHAR` column to an `INT` can force a scan. - Read the execution plan: look for table/index **scans** where a **seek** was expected, and for large discrepancies between estimated and actual row counts (usually stale statistics).

11. DDL / DML Basics

```
CREATE TABLE Sales.Summary (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Region VARCHAR(50) NOT NULL,
    Revenue DECIMAL(18,2) DEFAULT 0,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_T FOREIGN KEY (TerritoryID) REFERENCES Sales.SalesTerritory(TerritoryID),
    CONSTRAINT CK_Rev CHECK (Revenue >= 0)
);

INSERT INTO t (a, b) VALUES (1, 'x'), (2, 'y');
INSERT INTO t (a, b) SELECT a, b FROM other;
UPDATE t SET b = 'z' WHERE a = 1;
DELETE FROM t WHERE a = 1;
TRUNCATE TABLE t;           -- fast, no WHERE, minimal logging, resets IDENTITY

MERGE INTO target AS t
USING source AS s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.v = s.v
WHEN NOT MATCHED THEN INSERT (id, v) VALUES (s.id, s.v);
```

DELETE vs **TRUNCATE** vs **DROP** : delete rows (logged, can filter, keeps structure) / remove all rows (fast, no filter, keeps structure) / remove the table entirely.

12. Practice Checklist — AdventureWorks

- ☐ Total revenue by territory and year, only territories above a threshold
- ☐ Top 5 products by revenue **within each** category (**ROW_NUMBER** + CTE)
- ☐ Customers who have never placed an order (**NOT EXISTS**)
- ☐ Employees with their manager's name (self-join)
- ☐ YoY growth % per territory (**LAG**)
- ☐ Running total of sales by date (framed window)
- ☐ Each product's price vs its subcategory average (correlated subquery, then again as a window)
- ☐ Pivot years into columns using conditional aggregation
- ☐ Recursive CTE over the employee hierarchy with depth
- ☐ Take a query using **YEAR(col) = 2014** and rewrite it sargably
- ☐ Explain the difference between **WHERE** and **HAVING** with an example of each